

Convolution and Pooling as an Infinitely Strong Prior

- The **Prior probability distribution** is a probability distribution over the parameters of a model that encodes our beliefs about what models are reasonable, before we have seen any data.
- Priors can be considered **weak or strong** depending on how concentrated the probability density in the prior is.
- A **weak prior** is a prior distribution with high entropy, such as a Gaussian distribution with high variance.
- Such a prior allows the data to move the parameters more or less freely.
- A **strong prior** has very low entropy, such as a Gaussian distribution with low variance.
- Such a prior plays a more active role in determining where the parameters end up.
- An **infinitely strong prior** places zero probability on some parameters and says that these parameter values are completely forbidden, regardless of how much support the data gives to those values.

Convolution and Pooling as an Infinitely Strong Prior

- Network architectures certainly reflect strong prior beliefs about the nature of the problem they will be applied to.
- In CNN the weight sharing across space (in the case of images) and time (in the case of speech), enforces that you expect a space and time invariant input.
- An MLP with a certain number of layers, sizes, and choice of non-linearities, can only represent certain types of function.
- For eg. a single layer network isn't able to represent the XOR function. That's a **strong prior**.
- To generalize this example a bit, a network with only a few layers simply cannot learn a highly complex function.
- Network initialization values are a **weak prior**, because they get completely wiped out during training.
- At test time, they play almost no role in determining the inference outputs

Convolution and Pooling as an Infinitely Strong Prior

- Assumptions about the weights (before learning) in terms of acceptable values and range are encoded into the prior distribution of the weights.
- A weak prior is has a high variance and shows that there is **low confidence in the initial value of the weight**.
- A strong prior is turn shows a range of values about which we are **confident** before learning begins.
- An infinitely strong prior demarcates certain values as forbidden completely assigning them zero probability.



Convolution and Pooling as an Infinitely Strong Prior

- We can imagine a convolutional net as being similar to a fully connected net, but with an infinitely strong prior over its weights.
- This infinitely strong prior says that the weights for one hidden unit must be identical to the weights of its neighbor, but shifted in space.
- The prior also says that the weights must be zero, except for in the small, spatially contiguous receptive field assigned to that hidden unit.
- Overall, we can think of the use of convolution as introducing an infinitely strong prior probability distribution over the parameters of a layer.
- This prior says that the function the layer should learn contains only local interactions and is equivariant to translation.
- Likewise, the use of pooling is an infinitely strong prior that each unit should be invariant to small translations.

Convolution and Pooling as an Infinitely Strong Prior

- One key insight is that **convolution and pooling can cause underfitting.**
- Like any prior, **convolution and pooling are only useful when the assumptions made by the prior are reasonably accurate.**
- If a task relies on preserving precise spatial information, then using pooling on all features can increase the training error.
- When a task involves incorporating information from very distant locations in the input, then the prior imposed by convolution may be inappropriate.
- Conv layer: the function the layer should learn contains only local interaction and is equivariant to translation. 
- Pooling layer: each unit should be invariant to small translation.

Efficient Convolution Algorithms

- Modern convolutional network applications often involve networks containing more than one million units.
- Powerful implementations exploiting parallel computation resources are essential.
- However, in many cases it is also possible to speed up convolution by selecting an appropriate convolution algorithm.
- Convolution is equivalent to converting both the input and the kernel to the frequency domain using a Fourier transform, performing point-wise multiplication of the two signals, and converting back to the time domain using an inverse Fourier transform.
- For some problem sizes, this can be faster than the naive implementation of discrete convolution.

Efficient Convolution Algorithms

- When a d -dimensional kernel can be expressed as the outer product of d vectors, one vector per dimension, the kernel is called separable.
- When the kernel is separable, naive convolution is inefficient.
- It is equivalent to compose d one-dimensional convolutions with each of these vectors.
- The composed approach is significantly faster than performing one d -dimensional convolution with their outer product.
- The kernel also takes fewer parameters to represent as vectors.
- If the kernel is w elements wide in each dimension, then naive multidimensional convolution requires $O(w^d)$ runtime and parameter storage space, while separable convolution requires $O(w \times d)$ runtime and parameter storage space.
- Of course, not every convolution can be represented in this way.

Efficient Convolution Algorithms

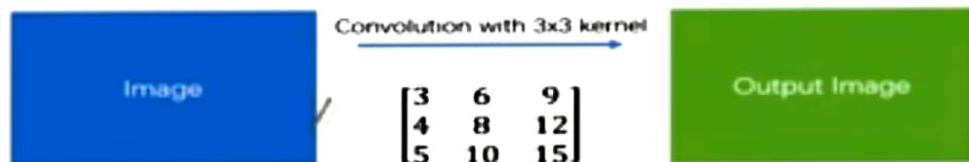
- A spatial separable convolution simply divides a kernel into two, smaller kernels.
- The most common case would be to divide a 3x3 kernel into a 3x1 and 1x3 kernel, like so:

$$\begin{bmatrix} 3 & 6 & 9 \\ 4 & 8 & 12 \\ 5 & 10 & 15 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

- Now, instead of doing one convolution with 9 multiplications, we do two convolutions with 3 multiplications each (6 in total) to achieve the same effect. With less multiplications, computational complexity goes down, and the network is able to run faster.

Efficient Convolution Algorithms

Simple Convolution



Spatial Separable Convolution



- The main issue with the spatial separable convolution is that not all kernels can be “separated” into two, smaller kernels.